Lecture 8: Continuation Value is All You Need

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 20, 2026

University of Toronto

Neural Value Function Iteration (nVFI)

Neural Value Function Iteration (nVFI)

Broadly:

1. Parameterize $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ using a neural network with params θ
2. Sample from Λ^{start} (somehow)
3. For each Λ^{start} compute the **outer Bellman residual** (defined below)
4. Update θ and repeat from Step 2 until convergence

Outer Bellman Residual

Consider the Krusell-Smith model.

Let $\widehat{V}_\theta^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ be a θ-parameterized neural network.

Informally, we want to satisfy as closely as possible,

$$\widehat{V}_\theta^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) - \mathbb{E}_{Z'}\left[\widehat{V}^{\text{start}}\left(x^{\text{end}}; \Lambda^{\text{start}\prime}, Z'\right) \mid Z\right] = 0,$$

where $\widehat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}\prime}, Z')$ is computed by **lookahead**, given $\widehat{V}_\theta^{\text{end}}, Z'$.

Conventions and Timing

Conventions

1. Household states do not change between the end of one period and the start of the next:

$$x_{it}^{\text{end}} = x_{i,t+1}^{\text{start}} \quad \text{and} \quad \Lambda_{it}^{\text{end}} = \Lambda_{i,t+1}^{\text{start}}.$$

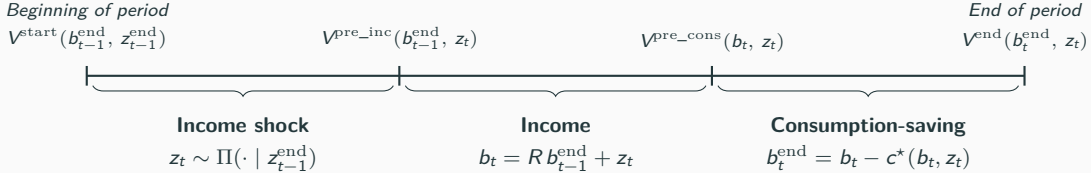
2. Aggregate shocks occur **between** household periods,
   - i.e. between $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ and $V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}\prime}, Z')$
3. A value function **slice** or **guess** depending only on **idiosyncratic** state is denoted by tilde,

$$\widetilde{V}^{\text{start}}(x^{\text{start}}), \quad \text{but} \quad V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z).$$

   - i.e. symbols such as $\widetilde{V}^{\text{start}}$ represent **function-valued variables**. More details below.

OLD Within-Period Timeline: Steady State

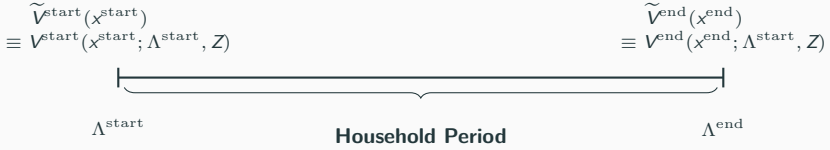
Recall the structure of timing within a period in the steady state version:



Or, simplified,

NEW Within-Period Timeline: Global Solution

With aggregate shocks, we rewrite:



In previous lectures, we learned how to map,

$$\left(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right) \mapsto \left(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}\right).$$

Let's call this $\Phi\left(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right) = \left(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}\right).$

Φ Concretely

This is quite abstract, but remember, if we discretize the household state, then,

$$\Phi\left(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right) = \left(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}\right),$$

is just a function from two **matrices and a scalar** to **two matrices**.

It might contain a loop for market clearing, but fundamentally it takes two **matrices and a scalar** as inputs and outputs **two matrices.**

Conventions: Dependence on Λ^{start}

- The model is deterministic within the household period.
- Thus, Λ^{end} is uniquely determined by Λ^{start}.
- Thus, it is **theoretically valid** to write $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$.

Note: I am **not** saying anything about how to compute or represent V^{end} on the computer yet!

This is purely the statement that $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ is **well-defined**.

Conventions: Partial Application of V^{end}

- Fixing Λ^{start} and Z, we can construct,

$$\widetilde{V}^{\text{end}}(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

- In computer-science terms, this is a **partial application** of V^{end} at $(\Lambda^{\text{start}}, Z)$.
- Let us write this as an operator taking a **function** to a **function**

$$\mathcal{V}(\Lambda^{\text{start}}, Z) = \widetilde{V}^{\text{end}}$$

- Seems abstract, but for us $\mathcal{V}$ is just a function from a **matrix and a scalar** to a **matrix**.
- Preview: $\mathcal{V}$ is the object we will attempt to approximate using a **neural network**.

Continuation Value Operator

- *CV* is the **continuation value operator**,

$$\mathcal{V}(\Lambda^{\text{start}}, Z) = \widetilde{V}^{\text{end}} \equiv V^{\text{end}}(\cdot; \Lambda^{\text{start}}, Z)$$

- It returns a **function**. That is, we may write,

$$\mathcal{V}(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

My view is that the **continuation value operator** $\mathcal{V}$ is the natural extension of the value function to HA models with aggregate shocks.

Continuation Value Is All You Need

Continuation Value Is All You Need

Suppose we have a model: within-period solver Φ, continuation value operator $\mathcal{V}$, aggregate shock process Π, and initial aggregate state $(\Lambda^{\text{start}}, Z)$.

Then we can simulate the model forward:

1. $\widetilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$ (Project continuation value)
2. $(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$ (Simulate to end of period)
3. Draw $Z' \sim \Pi(\cdot \mid Z)$ (Draw aggregate shock)
4. $(\Lambda^{\text{start}\prime}, Z') \equiv (\Lambda^{\text{end}}, Z')$ (Construct start of next period state)

Lookahead Given $\mathcal{V}$

Given $\mathcal{V}$, we can also perform **lookahead**.

1. $\widetilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$ (Project continuation value)
2. $(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$ (Simulate to end of period)
3. Draw $Z' \sim \Pi(\cdot \mid Z)$ (Draw aggregate shock)
4. $(\Lambda^{\text{start}\prime}, Z') \equiv (\Lambda^{\text{end}}, Z')$ (Construct start-of-next-period state)
5. $\widetilde{V}^{\text{end}\prime} = \mathcal{V}(\Lambda^{\text{start}\prime}, Z')$ (Project end-of-next-period value)
6. $(\widetilde{V}^{\text{start}\prime}, \Lambda^{\text{end}\prime}) = \Phi(\widetilde{V}^{\text{end}\prime}, \Lambda^{\text{start}\prime}, Z')$ (Iterate back to start-of-next-period value)
7. $\widetilde{V}^{\text{end,implied}} = \mathbb{E}_{Z'}[\widetilde{V}^{\text{start}\prime} \mid Z]$ ($\widetilde{V}^{\text{end}}$ should be equal to this)

We will be using this to generalize VFI.

OLD Typical Bellman Equation and Operator

The typical Bellman equation defines V via the relation,

$$V(b,z) = \max_{c \in [0,b]} \; u(c) + \beta \, \mathbb{E}\big[\, V(b',z') \,\big|\, z \,\big]$$

This can be thought of as a fixed point of the Bellman operator $\mathcal{T}$ which applies **lookahead**,

$$\mathcal{T}v(b,z) \equiv \max_{c \in [0,b]} \; u(c) + \beta \, \mathbb{E}\big[\, v(b',z') \,\big|\, z \,\big]$$

So that the Bellman equation is,

$$V = \mathcal{T}V.$$

I define a similar **outer Bellman equation**.

NEW Outer Bellman Equation

The true $\mathcal{V}$ also satisfies a recursive equation, the **outer Bellman equation:**

$$V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) \;=\; \mathbb{E}_{Z'}\left[\widetilde{V}^{\text{start}\prime}(x^{\text{end}}) \,\middle|\, Z\right],$$

where

$$\begin{aligned}
\widetilde{V}^{\text{end}} &= \mathcal{V}(\Lambda^{\text{start}}, Z) \\
\Lambda^{\text{end}} &= \Phi_\Lambda(\Lambda^{\text{start}}, \widetilde{V}^{\text{end}}, Z) \\
Z' &\sim \Pi_Z(\cdot \mid Z) \\
\widetilde{V}^{\text{end}\prime} &= \mathcal{V}(\Lambda^{\text{end}}, Z') \\
\widetilde{V}^{\text{start}\prime} &= \Phi_{\mathcal{V}}\big(\Lambda^{\text{end}}, \widetilde{V}^{\text{end}\prime}, Z'\big).
\end{aligned}$$

The Outer Bellman Operator $\mathcal{L}$

For any parametrized guess $\mathcal{V}_\theta$, we can define the **outer Bellman operator** $\mathcal{L}$ as:

$$\mathcal{L}\mathcal{V}_\theta(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv \mathbb{E}_{Z'}\left[\widetilde{V}^{\text{start}\prime}(x^{\text{end}}) \,\middle|\, Z\right],$$

where,

$$\begin{aligned}
\widetilde{V}^{\text{end}} &= \mathcal{V}_\theta(\Lambda^{\text{start}}, Z) \\
\Lambda^{\text{end}} &= \Phi_\Lambda(\Lambda^{\text{start}}, \widetilde{V}^{\text{end}}, Z) \\
Z' &\sim \Pi_Z(\cdot \mid Z) \\
\widetilde{V}^{\text{end}\prime} &= \mathcal{V}_\theta(\Lambda^{\text{end}}, Z') \\
\widetilde{V}^{\text{start}\prime} &= \Phi_V\big(\Lambda^{\text{end}}, \widetilde{V}^{\text{end}\prime}, Z'\big).
\end{aligned}$$

The Outer Bellman Residual

The true **continuation value operator** $\mathcal{V}$ is defined by,

$$\mathcal{L}\mathcal{V} = \mathcal{V}.$$

For any $\mathcal{V}_\theta$, we can define the **outer Bellman residual** as,

$$\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta,$$

and we can frame the problem of approximating $\mathcal{V}$ via $\mathcal{V}_\theta$ as,

$$\min_\theta \left\| \mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta \right\|^2.$$

Implementing CVIAYN

Training Loop

1. Sample $(\Lambda^{\text{start}}, Z)$ from ergodic distribution by simulating forward given $\mathcal{V}_\theta$.
2. Compute $\mathcal{L}V_\theta$ at each sample $(\Lambda_i^{\text{start}}, Z_i)$:
   - 2.1 Network predicts $\widehat{V}_\theta^{\text{end}}$ at $(\Lambda_i^{\text{start}}, Z_i)$.
   - 2.2 Simulate Λ_i^{start} forward to Λ_i^{end} using Φ.
   - 2.3 For each Z', predict $\widehat{V}^{\text{end}\prime}_\theta$ at $(\Lambda_i^{\text{end}}, Z')$ using $\mathcal{V}_\theta$.
   - 2.4 Iterate backward to get $V^{\text{start}\prime}$ using Φ.
   - 2.5 Take expectation $\widehat{V}_i^{\text{end}} \leftarrow \mathbb{E}_{Z'}\left[V^{\text{start}\prime} \mid Z\right]$.
3. Compute gradients of loss,

$$\frac{\partial}{\partial\theta}\sum_i \left\|\mathcal{V}_\theta(\Lambda_i^{\text{start}}, Z_i) - \widehat{V}_i^{\text{end}}\right\|^2,$$

taking $\widehat{V}_i^{\text{end}}$ as fixed labels, and update θ.

Φ: Already Done

As covered in previous notebooks, the function,

$$\Phi\left(\widetilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right) = \left(\widetilde{V}^{\text{start}}, \Lambda^{\text{end}}\right),$$

can be implemented with a chain of three HouseholdStages,

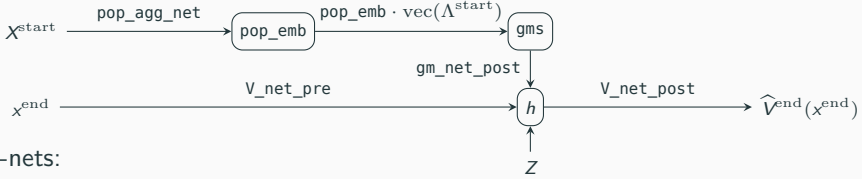
$$\text{MarkovStage} \circ \text{WealthChange} \circ \text{ConsumptionSavings}$$

acting on matrices representing the discretized idiosyncratic (b, z) grid:

- Iterate backward to get $\widetilde{V}^{\text{start}}$
- Iterate forward to get Λ^{end}.

Neural Network Architecture

We compress Λ^{start} using the method of (Han–Yang–E 2025), then combine with household-level features:



Four sub-nets:

1. pop_agg_net : $(b, z) \to \mathbb{R}^k$ — per-cell embedding aggregated against Λ^{start} to give the global moments $gms = \sum_{(b,z)} \Lambda(b, z) \cdot \texttt{pop\_agg\_net}(b, z) \in \mathbb{R}^k$.
2. V_pre : $(b, z) \to \mathbb{R}^m$ — cell-level features for the value head.
3. gm_post : $\mathbb{R}^k \to \mathbb{R}^m$ — projects *gms* into the value head's mid-space; added cell-wise to V_pre.
4. V_post : $\mathbb{R}^{m+N_Z} \to \mathbb{R}$ — final head; takes the summed features plus a one-hot of Z, outputs $\widehat{V}^{\text{end}}$.

Neural Network in Code

```
struct VNet{A, P, G, V}
    pop_agg_net :: A    # (b, z) → gm_dim         per-cell embedding for GM
    V_pre       :: P    # (b, z) → mid            per-cell features for V head
    gm_post     :: G    # gm_dim → mid            project GM to mid space
    V_post      :: V    # mid + N_Z → 1           combine + Z, read off V_end
end

function (net::VNet)Λ(::AbstractMatrix, Z_idx::Int, p::KSParams)
    pop_emb = net.pop_agg_net(HH_FEAT)          # (gm_dim, N_CELLS)
    gm      = pop_emb * Float32.(vecΛ())        # (gm_dim,)   pop-weighted average
    h_pre   = net.V_pre(HH_FEAT)                # (mid, N_CELLS)
    h_gm    = net.gm_post(gm)                   # (mid,)
    h       = h_pre .+ h_gm                     # broadcast → (mid, N_CELLS)
    pred    = net.V_post(vcat(h, Z_ROWS[Z_idx])) # (1, N_CELLS)
    return reshape(pred, p.N_b, length(p.z_grid))
end
```

The Lookahead Operator $\mathcal{L}V_\theta$

```
function lookahead(hh, vnet::VNet, Λ_start::AbstractMatrix, Z_idx::Int, p::KSParams)
    V_end_now = Float64.(vnetΛ(_start, Z_idx, p))
    env_now   = make_env(hh; ks_prices(integrate_K(hh, Λ_start), p.Z_vals[Z_idx], p)...)
    backward!(hh, V_end_now, env_now)                  # seats savings policyΛ
    _end = forward!(hh, Λ_start)

    V_end_label = zeros(Float64, p.N_b, length(p.z_grid))
    for Z_next in 1:N_Z
        V_start_next = inner_solve_backward(hh, vnet, Λ_end, Z_next, p)
        V_end_label .+= pΠ._Z[Z_idx, Z_next] .* V_start_next
    end
    return (; V_end_label, Λ_end)
end
```

Training Loss

Treating `target` $\equiv \widehat{V}_i^{\text{end}}$ as fixed labels, autodiff through `vnet`.

```
function build_targets(hh, vnet::VNet, batch, p::KSParams)
    targets = NamedTupleΛ{(:, :Z_idx, :target), Tuple{Matrix{Float64}, Int, Matrix{Float32}}}[]
    for Λ(_start, Z_idx) in batch
        out = lookahead(hh, vnet, Λ_start, Z_idx, p)
        push!(targets, (; Λ=copyΛ(_start), Z_idx, target=Float32.(out.V_end_label)))
    end
    return targets
end

function bellman_residual_loss(vnet::VNet, targets, p::KSParams)
    L = 0f0
    for Λ(, Z_idx, target) in targets
        pred  = vnetΛ(, Z_idx, p)
        scale = max(var(target), 1f-6)             # normalise by target scale
        L    += sum((pred .- target).^2) / (length(pred) * scale)
    end
    return L / length(targets)
end
```

Sampling Λ^{start}

MCMC-style: Simulate given the *current* V_θ, burn in, and subsample.

```
function sim_one_step(hh, vnet::VNet, Λ::AbstractMatrix, Z_idx::Int, p::KSParams)
    V_end = Float64.(vnetΛ(, Z_idx, p))
    env   = make_env(hh; ks_prices(integrate_K(hh, Λ), p.Z_vals[Z_idx], p)...)
    backward!(hh, V_end, env)
    return forward!(hh, Λ)
end

function sample_ergodic(hh, vnet::VNet, p::KSParams; T=200, burn=100, every=2, rng=MersenneTwister(8421))Λ
         = copyΛ(_ss)
    Z_idx = 1
    batch = Tuple{Matrix{Float64}, Int}[]
    for t in 1:TΛ
             = sim_one_step(hh, vnet, Λ, Z_idx, p)
        Z_idx = sample_Z(Z_idx, pΠ._Z, rng)
        if t > burn && (t - burn) % every == 0
            push!(batch, (copyΛ(), Z_idx))
        end
    end
    return batch
end
```

The training loop

```
function train!(vnet::VNet, hh, p::KSParams; epochs=50, lr=1e-3, T=200, burn=100, every=4)
    opt  = Flux.setup(Adam(lr), vnet)
    hist = Float64[]
    rng  = MersenneTwister(1729)
    for epoch in 1:epochs
        batch   = sample_ergodic(hh, vnet, p; T, burn, every, rng=MersenneTwister(rand(rng, UInt32)))
        targets = build_targets(hh, vnet, batch, p)
        loss, grads = withgradient(vnet) do net
            bellman_residual_loss(net, targets, p)
        end
        Flux.update!(opt, vnet, grads[1])
        push!(hist, Float64(loss))
    end
    return hist
end
```

Initialization and Pretraining

Pretrain on V_{ss} at perturbed Λ_{ss} and all Z.

```
function Λperturb_Λ(::AbstractMatrix, σ, rng)Λ
    p = Λ .* (1 .+ σ .* randn(rng, sizeΛ()))Λ
    p = maxΛ.(p, 0)
    return Λp ./ sumΛ(p)
end

function pretrain_to_ss!(vnet::VNet, V_ss_target, p::KSParams; epochs=300, lr=5e-3, Λn_=6, σ=0.10, seed=4747)
    opt    = Flux.setup(Adam(lr), vnet)
    target = Float32.(V_ss_target)
    rng    = MersenneTwister(seed)
    hist   = Float64[]
    for epoch in 1:epochsΛ
        _cloud = [Λperturb_Λ(_ss, σ, rng) for _ in 1:Λn_]
        loss, grads = withgradient(vnet) do net
            L = 0f0
            for Λ in Λ_cloud, Z_idx in 1:N_Z
                pred = netΛ(, Z_idx, p)
                L += sum((pred .- target).^2)
            end
            L / (Λn_ * N_Z * length(target))
```

Generalizations

Generalizations: Models

- General approach well-defined for any model for which you can implement Φ.
- If you can't implement Φ, then the issue does not lie with aggregate uncertainty.
- I provide `HouseholdStages.jl` to build high-performance household blocks.
- Can handle: spatial, multi-sector, OLG, search-matching, New Keynesian, frictional and durable assets (e.g. housing), rational inattention, Epstein-Zin preferences

Generalizations: Segmented Markets

Segmented-markets models, such as those with multiple locations or sectors, present additional challenges.

I develop:

- **Segmented Generalized Moments** to extend Han–Yang–E (2025) to model-consistently compress both segment (e.g. location) and aggregate data.
- **Lightweight Network Heads** method to speed up training of segmented-markets models with n segments and m household states from $O(nm)$ to effectively $O(n + m)$

Generalizations: Methods

The nVFI algorithm does not, in principle, depend on these implementation details:

- $\mathcal{V}_\theta$ parameterization: I use neural net, but method just requires some trainable function approximator.
- Value function representation: I use a matrix over discretized idiosyncratic state, but you could use splines, Chebyshev polynomials, or even another neural network.
- Population representation: I use a matrix over discretized idiosyncratic state, but you could use moments, finite agents, etc.
- Within-period solver: I use largely grid-based methods, but you could use EGM, Reiter, etc.

As long as you can implement the **within-period solver** Φ, this algorithm is well-defined.

Relationship to HouseholdStages.jl

The deep connection between this and my work on Household Stages is the following:

- Discrete-time HA models have a **compositional structure** based on **functional composition**
- By contrast, continuous-time HA models have a compositional structure based on **additive composition**.

The paper and package on Household Stages explores the implications of this for **within-period solvers** Φ

Informed by this, nVFI generalizes the Bellman Equation in terms of $\mathcal{V}$ and uses it for global solutions.